TOOLKIT · BACKEND ENGINEERING HANDBOOK

TERRAFORM COMPLETE HANDBOOK

Infrastructure as Code, HCL, State Management, Modules, Remote Backends, Security, CI/CD & Production Reliability

Phase	IV — Container Orchestration & Cloud Infrastructure
Tool ID	13 of 13
Curriculum Scope	Complete 3-Chunk Comprehensive Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/terraform/ (01, 02, and 03 markdown sources)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, DevOps Engineers, Cloud Platform Teams
Engineering Focus	IaC Declarative Lifecycle, State Ledger & Locking, Module Reusability, Environment Boundaries, Blast Radius, CI/CD Gateways, Drift & Incident Runbooks

This authoritative engineering handbook provides an exhaustive, production-grade guide to **Terraform** for backend developers and cloud infrastructure engineers. Spanning three comprehensive pedagogical layers—Mental Models, Practical Workflows, and Production Incident Mastery—it moves engineers from basic resource declarations to managing mission-critical cloud topologies safely. Readers master declarative HCL syntax, provider and resource graphs, state architecture and ledger mechanics, local versus remote backends with distributed state locking, modular infrastructure composition, environment partitioning strategies, workspaces vs. distinct configurations, resource address migrations with moved blocks, provider version pinning via .terraform.lock.hcl, blast radius minimization, Policy as Code guardrails, automated CI/CD validation and plan review pipelines, drift remediation, sensitive state governance, and real-world incident runbooks.

Curriculum Structure & Core Topics

Part I — Infrastructure as Code, Core Concepts & Mental Models: The IaC imperative, configuration-plan-apply-reconcile cycle, declarative vs imperative paradigms, providers and resource schemas, HCL fundamentals (blocks, arguments, expressions, variables, outputs, data sources), state ledger role, dependency graphs, drift detection, and local execution with terraform_data.

Part II — State Management, Modules, Environments & Safe Infrastructure Workflows: State management internals, local vs remote backends, distributed state locking and concurrency, reusable child module architecture, input/output composition, environment separation (directories vs workspaces), state migrations, moved blocks for refactoring, provider lock files, and safe state recovery.

Part III — Production Infrastructure, Security, CI/CD, Reliability & Incident Mastery: Production change control, blast radius containment, provider version pinning and upgrade paths, network isolation, short-lived workload identity, Policy as Code, cost optimization, multi-stage CI/CD pipelines, partial apply triage, prevent_destroy guardrails, drift investigation, and production incident runbooks.

Part I: Infrastructure as Code, Core Concepts & Mental Models

Chunk 1 of 3 · Backend Engineering Handbook

Terraform — Chunk 1 of 3: Infrastructure as Code, Core Concepts & Mental Models

Backend Engineering Tools & Infrastructure Handbook

Source file: `content/terraform/01-terraform-infrastructure-as-code-core-mental-models.md`

Learning objective: Understand how Terraform manages infrastructure declaratively, how configuration becomes real infrastructure, and how to use plans and state safely in backend engineering workflows.

Layer 1 — Fundamentals & Mental Models

1. What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool. It allows engineers to describe infrastructure using configuration files and then create, update, or destroy resources to match that configuration.

Instead of manually creating infrastructure through a cloud console, you define the desired configuration in files.

For example, a backend application might need:

- A virtual machine or container platform.
- A network and firewall rules.
- A managed PostgreSQL database.
- An object storage bucket.
- Identity and access permissions.
- Monitoring and supporting resources.

Terraform lets you express these resources in configuration, review proposed changes, and apply them through a repeatable workflow.

The core mental model

Configuration → Plan → Apply → Real infrastructure

Terraform compares the configuration with its understanding of the current infrastructure and calculates the changes needed to reconcile them.

```
Terraform configuration
          |
          v
   Initialize providers
          |
          v
   Read configuration
          |
          v
  Compare with state
  and remote objects
          |
          v
       Plan
          |
          v
 Review proposed changes
          |
          v
        Apply
          |
          v
 Infrastructure changes
          |
          v
 Update Terraform state
```

The plan is a proposed set of actions, not a guarantee that every action will succeed. Permissions, quotas, concurrent changes, API failures, and provider behavior can still cause an apply to fail.

2. Why Infrastructure as Code matters

Manual infrastructure creation creates operational risks:

- Environments drift apart.
- Engineers forget configuration details.
- Changes are difficult to review.
- Rebuilding an environment becomes slow.
- Resource ownership becomes unclear.
- Accidental deletion becomes harder to prevent.
- Auditability suffers.

Infrastructure as Code improves repeatability, reviewability, and recovery.

Consider two environments:

- Development uses one application instance and a small database.
- Production uses multiple application instances and a highly available database.

Both can use the same underlying module design while differing through explicit configuration.

Terraform does not automatically make environments identical or safe. The quality of the configuration, module design, access controls, and review process determines how reliable the workflow becomes.

3. Terraform versus related tools

Tool	Primary responsibility
Terraform	Provision and manage infrastructure resources
Docker	Package applications and their runtime dependencies
Kubernetes	Schedule and operate containerized workloads
Ansible	Configure systems and execute automation tasks
Packer	Build machine images
Maven	Build Java applications and manage dependencies
Git	Track configuration and application source changes

These tools can complement one another.

For example:

1. Maven builds a Spring Boot application.
2. Docker packages it into an image.
3. Terraform provisions the infrastructure.
4. Kubernetes runs the application.
5. Prometheus and Grafana support monitoring.
6. Git tracks changes across the repository.

Terraform can provision a Kubernetes cluster, but Kubernetes is responsible for managing workloads inside that cluster.

4. Declarative versus imperative thinking

An imperative instruction describes the steps to perform.

```
Create a server.
Install dependencies.
Configure the network.
Start the service.
```

A declarative configuration describes the intended result.

```
resource "example_server" "api" {
  name = "backend-api"
  size = "small"
}
```

This is illustrative HCL, not a working provider-specific resource.

Terraform evaluates the configuration, constructs a dependency graph, and determines the actions needed to approach the desired state.

Declarative configuration does not mean Terraform can infer every operational requirement. Engineers must still define networking, security, dependencies, sizing, and lifecycle behavior correctly.

5. Providers and resources

A **provider** is a plugin that lets Terraform interact with an external API or platform.

Examples include providers for cloud platforms, Kubernetes, DNS services, and monitoring systems.

A **resource** represents an object Terraform can create, read, update, or delete through a provider.

Conceptually:

```
provider "example" {
  region = "example-region"
}

resource "example_network" "main" {
  name = "backend-network"
}
```

The provider and resource types above are placeholders. Real configuration must use resource types supported by the selected provider.

Terraform uses provider schemas to validate configuration and communicate with the relevant platform APIs.

6. HCL: Terraform's configuration language

Terraform configurations are commonly written in HashiCorp Configuration Language (HCL).

Basic concepts include:

- **Blocks:** Define resources, providers, variables, outputs, and other configuration objects.
- **Arguments:** Assign values to configuration properties.
- **Expressions:** Calculate or reference values.
- **Variables:** Supply configurable inputs.
- **Outputs:** Expose selected values from a configuration.
- **Data sources:** Read information about existing objects.
- **Modules:** Package reusable Terraform configurations.

Example:

```
variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "development"
}

output "environment_name" {
  value = var.environment
}
```

The variable `environment` supplies an input value. The output exposes a value from the configuration.

Outputs can reveal sensitive information if configured carelessly. Marking an output as sensitive affects display behavior, but it does not automatically encrypt every stored copy of that value.

7. Resources, data sources, and modules

Resources

Resources describe infrastructure Terraform manages.

Conceptual example:

```
resource "example_bucket" "logs" {
  name = "backend-logs"
}
```

Data sources

Data sources read information about existing infrastructure or other external objects.

Conceptual example:

```
data "example_network" "existing" {
  name = "shared-network"
}
```

A data source does not ordinarily create the object it reads. It retrieves information for use elsewhere in the configuration.

Modules

A module is a reusable collection of Terraform configuration.

For example, a reusable backend module might define:

- Application compute resources.
- Network attachments.
- Health-check configuration.
- Monitoring integration.
- Security settings.

A root module composes resources and child modules into a deployable configuration.

Modules improve consistency when designed well, but abstraction can also hide important details. A module should expose the configuration consumers genuinely need without making operational behavior mysterious.

8. Terraform state: the most important concept

Terraform state maps Terraform-managed resource addresses to the real-world objects Terraform manages.

State helps Terraform understand which resource in the configuration corresponds to which remote object.

For example:

```
Terraform address:
example_server.api

Remote object:
server-83921
```

The address and identifier above are illustrative.

State supports:

- Tracking managed resources.
- Detecting changes.
- Determining which objects correspond to configuration.
- Calculating dependency-aware changes.
- Recording attributes returned by providers.

State is not simply a backup

State is not equivalent to a complete infrastructure backup. It may not contain all information necessary to reconstruct every external system or its data.

It can also contain sensitive values. Treat state files as sensitive operational data.

Local versus remote state

- **Local state:** Stored on the engineer's machine or in the working directory.
- **Remote state:** Stored in a supported remote backend.

For individual experiments, local state can be convenient. For collaborative environments, remote state generally provides a stronger foundation for shared workflows.

A production remote-state design should consider access control, encryption, locking or concurrency controls where supported, versioning or recovery options, and backup procedures.

9. The dependency graph

Terraform builds a dependency graph from references and explicit dependency declarations.

Suppose an application server requires a network:

```
Network
   |
   v
Application server
   |
   v
Application configuration
```

Terraform can determine that the network must be created before resources that depend on it.

Referencing another resource's attributes normally creates an implicit dependency.

depends_on can declare an explicit dependency when the relationship is real but is not otherwise expressed through configuration references.

Avoid adding unnecessary dependencies. Overly restrictive dependency graphs can reduce parallelism and make configuration harder to maintain.

10. Desired state, drift, and reconciliation

Drift occurs when real infrastructure no longer matches Terraform's expected configuration and state.

Examples include:

- An engineer manually changes a firewall rule.
- A resource is deleted through a cloud console.
- An external system modifies a managed object.
- Infrastructure configuration changes without a corresponding Terraform update.

A Terraform plan can help identify proposed corrective actions, although the exact result depends on provider refresh behavior, state, configuration, and the resource's lifecycle semantics.

Do not automatically apply every plan. First determine whether the difference is intentional, accidental, or caused by an incomplete understanding of the infrastructure.

Layer 2 — Practical Engineering & Workflows

11. Install and initialize Terraform

Install Terraform using the official distribution or an approved package manager for your operating system.

Verify the installation:

```
terraform version
```

Create a project directory:

```
mkdir backend-infrastructure
cd backend-infrastructure
```

Create `main.tf` and add a small local-only configuration.

```
terraform {
  required_version = ">= 1.5.0"
}

resource "terraform_data" "example" {
  input = {
    application = "backend-api"
    environment = "development"
  }
}

output "example_input" {
  value = terraform_data.example.output
}
```

`terraform_data` is a built-in resource available in modern Terraform versions. It lets you practice Terraform's lifecycle without provisioning a paid cloud resource.

The example's version constraint is a minimum-version constraint, not a complete dependency-pinning strategy for every team.

Initialize:

```
terraform init
```

Initialization prepares the working directory, installs required providers where applicable, and initializes the configured backend.

For this local-only example, there is no external cloud provider to configure.

12. Format and validate configuration

Format the configuration:

```
terraform fmt
```

Check configuration syntax and internal consistency:

```
terraform validate
```

Formatting makes files consistent. Validation catches configuration problems that Terraform can identify without performing the full infrastructure change.

Neither command proves that a production deployment will succeed.

For example, a syntactically valid cloud configuration may still fail because:

- The identity lacks permissions.
- A region does not support the requested resource.
- A quota has been reached.
- A network range overlaps with an existing network.
- A provider API is unavailable.

13. Plan before applying

Generate a plan:

```
terraform plan
```

Terraform shows the proposed actions, such as creating, updating, or deleting resources.

Common plan symbols include:

- `+` — create.
- `~` — update in place.
- `-` — destroy.
- `-/+` — destroy and recreate, in configurations where replacement is required.

Always inspect the plan for destructive changes.

For a saved plan:

```
terraform plan -out=tfplan
terraform show tfplan
```

Apply the saved plan:

```
terraform apply tfplan
```

Saved plans help separate review from execution. However, they are operational artifacts that may contain sensitive data. Protect them, avoid committing them, and regenerate them if relevant infrastructure or configuration assumptions have changed.

For a disposable local exercise, a direct `terraform apply` can be convenient after reviewing the proposed changes.

14. Apply and inspect the result

Apply the local example:

```
terraform apply
```

Review the proposed changes and confirm when prompted.

Then inspect the state:

```
terraform state list
terraform state show terraform_data.example
```

Inspect outputs:

```
terraform output
```

The resource should appear in the state, and the output should expose the configured example data.

Destroy the disposable exercise when finished:

```
terraform destroy
```

Review the destruction plan and confirm it.

Even local exercises deserve careful cleanup habits. In cloud configurations, `destroy` can remove important resources and may cause irreversible data loss.

15. Variables and environment-specific configuration

As a configuration grows, hard-coded values become difficult to manage.

Create `variables.tf`:

```
variable "application_name" {
  description = "Name of the application"
  type        = string
}

variable "environment" {
  description = "Deployment environment"
  type        = string

  validation {
    condition = contains(
      ["development", "staging", "production"],
      var.environment
    )
    error_message = "Choose development, staging, or production."
  }
}
```

Use the variables in `main.tf`:

```
resource "terraform_data" "application" {
  input = {
    name        = var.application_name
    environment = var.environment
  }
}
```

Provide values through a `.tfvars` file:

```
application_name = "orders-api"
environment      = "development"
```

For example, save the values in `development.tfvars` and run:

```
terraform plan -var-file="development.tfvars"
```

Use the same variable interface across environments where practical, while keeping environment-specific values explicit.

Do not store credentials or production secrets in ordinary `.tfvars` files committed to Git.

16. Outputs and data flow

Outputs expose selected values from a module.

```
output "application_name" {
  description = "Configured application name"
  value       = var.application_name
}
```

Outputs are useful when:

- A root module needs to expose a resource identifier.
- A child module returns information to its caller.
- A deployment workflow needs a value produced by Terraform.

Use outputs deliberately. Avoid exposing unnecessary internal details or sensitive values.

A downstream system should not blindly treat an output as a secret-management mechanism.

17. Terraform workflow in Git

A typical repository might look like this:

```
infrastructure/
■■■ main.tf
■■■ variables.tf
■■■ outputs.tf
■■■ versions.tf
■■■ development.tfvars
■■■ staging.tfvars
■■■ README.md
```

A practical review workflow:

1. Create a branch for the infrastructure change.
2. Update Terraform configuration.
3. Run `terraform fmt -check`.
4. Run `terraform validate`.
5. Generate and inspect a plan.
6. Open a pull request with the purpose and impact of the change.
7. Run an approved plan workflow in CI.
8. Review the resulting plan before applying.
9. Apply through the team's authorized deployment process.
10. Verify the real infrastructure after deployment.

The plan is an important review artifact, but the final result should also be validated against the actual environment.

18. Common Terraform commands

Command	Purpose
`terraform init`	Initialize the working directory and backend
`terraform fmt`	Format Terraform files
`terraform validate`	Validate configuration
`terraform plan`	Preview proposed changes
`terraform apply`	Apply infrastructure changes
`terraform destroy`	Destroy managed resources covered by the configuration
`terraform output`	Display root-module outputs
`terraform state list`	List resource addresses in state
`terraform state show`	Inspect one state resource
`terraform show`	Display a plan or state representation

Command	Purpose
`terraform providers`	Show provider requirements
`terraform graph`	Produce a dependency graph representation

Some commands require initialized configuration, provider access, state, or other context. Their output is not a substitute for understanding the current environment.

Layer 3 — Production Engineering & Interview Mastery

19. State safety and collaboration

When multiple engineers work on the same infrastructure, state management becomes a shared operational concern.

What can go wrong?

Two engineers may attempt conflicting changes at the same time. One engineer's plan may become stale after another engineer applies a change. A local state file may diverge from the real infrastructure. A state file may be lost or exposed.

Safer practices

- Use an appropriate remote backend for shared environments.
- Use supported state locking or concurrency controls.
- Restrict state access to authorized identities.
- Protect state backups and versions.
- Avoid manual state-file editing.
- Review plans before applying.
- Avoid concurrent applies to the same state unless the workflow explicitly supports them.
- Keep infrastructure changes auditable.

Remote state alone does not guarantee safe collaboration. Access controls, locking behavior, backup procedures, and release coordination all matter.

20. Drift investigation

Scenario: Someone manually modifies a network rule outside Terraform.

A disciplined response:

1. Confirm the affected environment and resource.
2. Inspect the current infrastructure through the appropriate provider tooling.
3. Run a fresh Terraform plan.
4. Determine whether the manual change was intentional.
5. If the configuration is authoritative, update it to represent the intended result.

6. If the manual change was unauthorized or accidental, prepare a reviewed corrective plan.
7. Apply only after verifying the impact.
8. Investigate why the out-of-band modification occurred.

Avoid immediately running `terraform apply` without reviewing the proposed changes. The plan may contain unrelated modifications or destructive replacements.

21. Resource replacement and lifecycle risk

Some resource changes can be performed in place. Others require destroying and recreating the resource.

A replacement can cause:

- A new resource identifier.
- Temporary downtime.
- Loss of data for disposable resources.
- Dependency changes.
- A temporary increase in infrastructure cost.
- Failed deployments if dependent systems expect the old identifier.

Before applying a replacement, ask:

1. Is the resource stateful?
2. Will data be preserved?
3. Can the service tolerate interruption?
4. Do dependent resources need updates?
5. Is a staged migration possible?
6. Is there a tested recovery path?

Terraform lifecycle configuration can influence replacement behavior, but lifecycle flags are not substitutes for an explicit migration plan.

22. Importing existing infrastructure

An organization may already have manually created infrastructure that should become Terraform-managed.

The general process is:

1. Write configuration that describes the existing object.
2. Identify the correct Terraform resource address.
3. Use the provider's supported import mechanism.
4. Inspect the resulting state.
5. Generate a plan.
6. Refine the configuration until the plan reflects the intended state.

Modern Terraform supports configuration-driven imports, and individual providers may have specific import requirements.

Importing is not the same as automatically generating a perfect configuration. The configuration must still describe the intended infrastructure accurately.

Never import a resource into a configuration without checking ownership and the potential for duplicate management.

23. Secrets and sensitive data

Terraform configurations and state may contain credentials, tokens, connection strings, or other sensitive attributes.

Safer handling includes:

- Obtain credentials through an approved identity mechanism.
- Prefer short-lived credentials where supported.
- Keep secret values out of source control.
- Restrict access to state and plan artifacts.
- Use the sensitive-value mechanisms supported by Terraform.
- Integrate with an approved secret manager when appropriate.
- Rotate credentials when exposure is suspected.

A value marked `sensitive` is primarily subject to redaction in selected output contexts. This does not automatically prevent the value from being stored in state or plan files.

Do not assume that hiding a value in terminal output makes it safe to persist.

24. CI/CD and controlled infrastructure changes

A production infrastructure pipeline should separate validation, review, and execution.

```
Pull request
     |
     v
Format and validate
     |
     v
Generate plan
     |
     v
Review changes
     |
     v
Authorized approval
     |
     v
Apply approved changes
     |
     v
Verify infrastructure
```

The exact pipeline depends on the organization's deployment platform and security requirements.

Good controls include:

- Separate identities for plan and apply where appropriate.
- Least-privilege permissions.
- Protected production environments.
- Auditable approvals.
- Safe handling of saved plans.
- Drift monitoring.

- Explicit destruction safeguards.
- Recovery procedures for failed applies.

A successful Terraform apply is not the end of the workflow. Confirm that the application can actually use the provisioned infrastructure.

25. Production incident simulation: a plan wants to destroy a database

Scenario: A routine pull request produces a plan showing database replacement.

Response

1. Stop the apply.
2. Identify the exact resource and proposed replacement reason.
3. Inspect configuration changes, provider behavior, and immutable attributes.
4. Determine whether the resource contains persistent or business-critical data.
5. Check whether a replacement is intentional.
6. If unexpected, correct the configuration or revise the migration strategy.
7. Regenerate the plan.
8. Obtain the appropriate review before applying.

Do not assume that Terraform understands the business impact of a resource replacement. It understands the provider's resource model, not the importance of the database to your customers.

26. Production incident simulation: state lock or concurrent apply

Scenario: An apply fails because state is locked, or the team suspects two workflows are changing the same environment.

Response

1. Identify the backend and locking mechanism.
2. Find the workflow or engineer holding the lock.
3. Confirm whether an apply is still running.
4. Review recent pipeline and backend logs.
5. Allow the active operation to finish if it is healthy.
6. Remove a stale lock only after confirming it is genuinely stale and following the backend's supported procedure.
7. Re-run the plan against current state.

Never force-unlock state merely to make an error disappear. If another apply is active, removing the lock can permit conflicting operations and corrupt the team's understanding of infrastructure state.

27. SDE-1 interview questions and answers

1. What is Infrastructure as Code?

It is the practice of defining and managing infrastructure through version-controlled configuration rather than relying solely on manual actions.

2. What does Terraform do?

It evaluates infrastructure configuration, plans changes, and uses providers to manage supported resources.

3. What is a Terraform provider?

A plugin that lets Terraform interact with an external platform or API.

4. What is a resource?

A Terraform-managed object such as a server, network, database, or supported platform object.

5. What is a data source?

A configuration object used to read information about existing infrastructure or other external data.

6. What is Terraform state?

A record that maps Terraform resource addresses to real-world objects and stores attributes needed for Terraform's management workflow.

7. Why is state important?

It helps Terraform identify managed resources and calculate changes. Losing or mishandling state can make infrastructure management unsafe.

8. What is the difference between `plan` and `apply`?

`plan` previews proposed actions. `apply` executes the selected changes.

9. What is infrastructure drift?

A difference between the actual infrastructure and the intended or tracked configuration.

10. What is a Terraform module?

A reusable collection of Terraform configuration that packages infrastructure logic behind an interface.

11. Why use variables?

Variables make configuration reusable and allow values to be supplied for different environments without duplicating entire configurations.

12. Why use remote state?

It supports a shared state workflow and can provide centralized access control, recovery, and locking or concurrency controls when supported.

13. Why should state files be treated as sensitive?

They may contain sensitive infrastructure attributes or secret values, depending on the resources and configuration.

14. Does `terraform validate` prove an infrastructure deployment will succeed?

No. It validates configuration properties that Terraform can check locally. Permissions, quotas, API failures, and environment-specific issues can still prevent deployment.

15. What is a dangerous Terraform plan?

A plan containing unexpected destructive actions, replacements, security changes, or modifications with unacceptable operational impact.

16. What happens when a resource requires replacement?

Terraform may need to destroy and recreate it. This can change identifiers, interrupt service, or risk data loss.

17. What does `terraform init` do?

It initializes the working directory, installs required providers, and prepares the configured backend.

18. How do you safely adopt manually created infrastructure?

Write matching configuration, import the object through supported mechanisms, inspect state, and review plans until Terraform's proposed actions match the intended result.

19. Is marking a variable sensitive enough to secure a secret?

No. It can redact selected output, but state and plan artifacts may still contain the value. Access control and appropriate secret management are also required.

20. How would you respond to a plan that unexpectedly destroys a production database?

Stop, investigate the replacement reason and configuration, assess data and availability risk, correct the design, regenerate the plan, and require an appropriate review before applying.

---

28. Hands-on mastery assessment

Complete this exercise using the local `terraform_data` resource before provisioning any cloud infrastructure.

- [ ] Create a Terraform project.
- [ ] Initialize the working directory.
- [ ] Define typed input variables.
- [ ] Add validation to at least one variable.
- [ ] Create a resource that uses the variables.
- [ ] Add an output.
- [ ] Run formatting and validation.
- [ ] Generate and review a plan.
- [ ] Apply the plan.
- [ ] Inspect the resulting state.
- [ ] Change a variable and review the next plan.
- [ ] Destroy the disposable resource after reviewing the plan.
- [ ] Explain why remote state and state access controls matter.
- [ ] Explain how you would respond to an unexpected destructive plan.

Mastery standard

You should be able to explain Terraform's lifecycle without memorizing commands, identify state as a critical operational asset, recognize destructive changes, and describe how to move from local practice to a reviewed production workflow.

Terraform Chunk 1 completion protocol

Save this chapter as:

`content/terraform/01-terraform-infrastructure-as-code-core-mental-models.md`

Terraform has **three planned content chunks**. Do not generate a PDF yet.

The completed Terraform handbook must combine all three chapters into exactly one final PDF:

`docs/terraform/BEH_TERRAFORM_COMPLETE_HANDBOOK.pdf`

After all three Terraform chapters are saved and verified:

1. Generate the complete Terraform handbook PDF.
2. Verify content order, code blocks, tables, and page rendering.
3. Update the README with verified progress.
4. Commit the completed Terraform sources and verified PDF using the exact commit message:

`Terraform done`

Do not create a PDF for this chunk alone. The full repository still requires all 13 individual tool handbooks and one master handbook.

Part II: State Management, Modules, Environments & Safe Infrastructure Workflows

Chunk 2 of 3 · Backend Engineering Handbook

Terraform — Chunk 2 of 3: State Management, Modules, Environments & Safe Infrastructure Workflows

Backend Engineering Tools & Infrastructure Handbook

Source file: `content/terraform/02-terraform-state-modules-environments-safe-workflows.md`

Learning objective: Learn to manage Terraform state safely, structure reusable infrastructure modules, separate environments, and collaborate on infrastructure changes without creating unnecessary production risk.

Layer 1 — Fundamentals & Mental Models

1. Why Terraform state deserves special attention

In the first chapter, we established the basic lifecycle:

Configuration → Plan → Apply → Infrastructure

Now we need to understand how Terraform remembers which resources it manages.

Consider a configuration that declares a database:

```
resource "example_database" "orders" {
  name = "orders-db"
}
```

Terraform needs to associate the resource address `example_database.orders` with the actual database created through the provider.

That relationship is recorded in Terraform state.

State can contain resource identifiers, provider-returned attributes, dependency information, and potentially sensitive values. It is therefore a critical operational asset.

The three things to distinguish

Concept	Meaning
Configuration	What you declare Terraform should manage
State	Terraform's record of the resources it manages
Real infrastructure	The actual objects and their current behavior

These are related, but they are not interchangeable.

A configuration file is not a state backup. State is not a complete backup of a database's contents. And neither one guarantees that the real infrastructure is healthy.

2. How Terraform uses state

Imagine this sequence:

1. You declare an application server.
2. Terraform creates it.
3. The provider returns the server's identifier and attributes.
4. Terraform records the association in state.
5. On a later plan, Terraform uses configuration, state, and provider observations to calculate proposed changes.

The state helps Terraform understand resource identity and prior management decisions.

Without the correct state, Terraform may not know that an existing object corresponds to a declared resource. Depending on the situation, this can lead to unexpected creation attempts, conflicts, or unsafe plans.

Mental model: State is Terraform's management ledger, not the infrastructure itself.

3. Local state versus remote state

Local state

By default, a simple Terraform project may store state in a local file such as `terraform.tfstate`.

Advantages:

- Simple to understand.
- Convenient for disposable local exercises.
- No remote backend setup required.

Risks:

- Other engineers cannot automatically share the same state safely.
- A lost workstation can mean lost state.
- Local state may be accidentally committed or exposed.
- Concurrent workflows are difficult to coordinate.

Remote state

A remote backend stores state using a supported remote storage mechanism.

Benefits can include:

- Centralized access control.
- Shared state across authorized team members.
- State locking or concurrency controls where supported.
- Versioning and recovery options when configured.
- Integration with controlled deployment pipelines.

Remote state does not automatically guarantee all these benefits. The selected backend and its configuration determine which capabilities are available.

Choosing an approach

Scenario	Typical choice
Disposable local learning exercise	Local state
Team-managed shared environment	Remote state
Production infrastructure	Remote state with access controls, concurrency protection, and recovery planning

4. State locking and concurrent operations

Suppose two engineers try to apply changes against the same state at nearly the same time.

Engineer A plans to update a network. Engineer B plans to change the same environment before A's operation finishes.

Without suitable coordination, concurrent changes can produce conflicts or leave engineers with stale assumptions.

Where supported, state locking prevents multiple Terraform operations from simultaneously modifying the same state.

However, locking is not a substitute for change review. It coordinates state operations; it does not determine whether a proposed change is correct.

Important: Do not assume every backend supports locking in the same way. Check the chosen backend's documented capabilities and configuration.

5. Terraform modules: reusable infrastructure design

A module is a collection of Terraform configuration files that work together as a reusable unit.

Every Terraform working directory containing configuration is a root module. A root module can call child modules.

For example, an organization might define:

```
infrastructure/
■■■ main.tf
■■■ variables.tf
■■■ outputs.tf
■■■ modules/
    ■■■ network/
    ■■■ application/
    ■■■ database/
```

Each module has a defined responsibility.

- `network` manages networking resources.
- `application` manages application compute and related settings.
- `database` manages database infrastructure.

The directory structure is illustrative. The actual resources depend on the provider and platform.

Why modules matter

Without modules, teams may duplicate similar configuration across multiple environments. Duplication makes it easier for security settings, tags, resource sizing, or networking rules to drift apart.

Modules allow teams to centralize common patterns and expose inputs for legitimate differences.

The abstraction tradeoff

A module that hides every detail may become difficult to troubleshoot. A module that exposes every underlying resource property may not provide much abstraction.

A good module offers:

- A clear purpose.
- Explicit inputs.
- Useful outputs.
- Sensible defaults.
- Documented security and lifecycle behavior.
- A manageable interface.

6. Module inputs and outputs

A module can accept inputs through variables and return selected information through outputs.

Conceptual root-module configuration:

```
module "application" {
  source = "./modules/application"

  application_name = "orders-api"
  environment      = "development"
}
```

The child module might define:

```
variable "application_name" {
  type = string
}

variable "environment" {
  type = string
}

output "application_name" {
  value = var.application_name
}
```

These snippets illustrate the module interface; a real module must also define the resources it manages.

Module outputs can be referenced by the root module:

```
output "deployed_application_name" {
  value = module.application.application_name
}
```

Keep outputs intentional. Do not expose credentials or unnecessary internal details merely because a child module can return them.

7. Separate environments without losing consistency

A backend project may have development, staging, and production environments.

They often share architectural patterns but differ in:

- Resource size.
- Replica count.
- Network boundaries.
- Access permissions.
- Availability requirements.
- Data classification.
- Backup and recovery settings.
- Cost constraints.

The goal is **consistent design with explicit environment-specific configuration**.

Common approaches

Separate directories or root modules

```
environments/
■■■ development/
■■■ staging/
■■■ production/
```

Each environment can have its own configuration and state boundary.

Reusable modules with separate root configurations

Each environment calls the same modules but supplies different values.

Terraform workspaces

Workspaces provide separate state instances associated with a configuration and backend. They can be useful for certain workflows, but they do not automatically provide strong security or organizational isolation.

For critical environments, separate configurations, backend state boundaries, permissions, and deployment controls may be more appropriate than relying on workspace names alone.

8. Terraform workspaces: what they do and do not do

Terraform workspaces allow multiple state instances to be associated with the same configuration.

Conceptually:

```
Same configuration
       |
       +---- development state
       |
       +---- staging state
       |
       +---- production state
```

The exact behavior depends on the backend and workspace configuration.

Workspaces do not automatically create:

- Separate cloud accounts.
- Different IAM permissions.
- Network isolation.
- Independent deployment approval policies.
- Separate credentials.
- Protection against applying to the wrong environment.

A workspace is a state-management mechanism, not a complete environment-security boundary.

Before using workspaces for production, determine whether they fit the organization's access-control, isolation, and recovery requirements.

9. Dependencies and module composition

A root module can connect child modules through their inputs and outputs.

For example:

```
Network module
      |
      v
Database module

Network module
      |
      v
Application module
      |
      v
Application endpoint output
```

When a module input references another module's output, Terraform can establish an implicit dependency.

Avoid hard-coding identifiers that should be supplied by the infrastructure graph. Explicit references make relationships easier to understand and reduce accidental coupling to a particular environment.

Do not add `depends_on` mechanically. Use it when a genuine dependency exists but is not already represented through expressions.

Layer 2 — Practical Engineering & Workflows

10. Configure a remote backend safely

Terraform supports multiple backends, each with its own requirements and capabilities.

A remote backend configuration may look conceptually like this:

```
terraform {
  backend "example" {
    # Backend-specific configuration goes here.
  }
}
```

`example` is a placeholder, not a real backend type. Replace it with a supported backend and its documented settings.

Backend configuration is special: Terraform must initialize the backend before normal planning and applying.

A safe setup sequence is:

1. Choose the backend.
2. Create its storage and access controls through an approved process.
3. Configure locking or concurrency protection if supported.
4. Configure encryption and recovery options where available.
5. Add the backend block to the Terraform configuration.
6. Run `terraform init`.
7. Review any state migration prompts carefully.
8. Verify that the intended state is available.
9. Test the normal plan workflow before making infrastructure changes.

Never delete a local state file just because a remote backend has been configured. First verify that the state migration or initialization completed correctly and that the remote state contains the expected resource mappings.

11. Inspect state safely

Useful commands include:

```
terraform state list
terraform state show <resource-address>
terraform show
```

For example:

```
terraform state list
terraform state show terraform_data.application
```

These commands help inspect what Terraform currently tracks.

State inspection can reveal sensitive values. Be careful when sharing terminal output in issue trackers, pull requests, chat, or screenshots.

Do not edit state files manually

Terraform provides specific state commands for supported management tasks. Directly editing a state file can corrupt resource mappings or introduce inconsistencies.

If state must be repaired, investigate the supported operation for the situation and take a recovery backup where appropriate.

12. Recovering from state problems

State problems can arise from:

- Accidental deletion.
- Incorrect backend configuration.
- Failed state migration.
- Resource changes made outside Terraform.
- A resource imported incorrectly.
- An operation interrupted during a critical transition.

A disciplined recovery workflow:

1. Stop concurrent applies.
2. Identify the correct environment and backend.
3. Inspect available state versions or backups.
4. Determine whether real infrastructure is still present.
5. Preserve the current evidence before attempting repair.
6. Follow the backend's recovery procedure.
7. Run a fresh plan.
8. Review all proposed changes before applying.

Do not blindly copy an old state file over the current state. The infrastructure may have changed since the backup was created.

13. Build a reusable application module

Create a module directory:

```
mkdir -p modules/application
```

Create `modules/application/variables.tf`:

```
variable "application_name" {
  description = "Application identifier"
  type        = string
}

variable "environment" {
  description = "Deployment environment"
  type        = string
}
```

Create `modules/application/main.tf`:

```
resource "terraform_data" "application" {
  input = {
    application_name = var.application_name
    environment      = var.environment
  }
}
```

Create `modules/application/outputs.tf`:

```
output "application_name" {
  value = var.application_name
}

output "environment" {
  value = var.environment
}
```

Now configure the root module in `main.tf`:

```
module "application" {
  source = "./modules/application"

  application_name = "orders-api"
  environment      = "development"
}

output "application_name" {
  value = module.application.application_name
}
```

This local exercise does not create cloud infrastructure. It teaches module boundaries, inputs, outputs, and composition using Terraform's built-in `terraform_data` resource.

Initialize and validate:

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
```

Review the plan before confirming the apply.

14. Use environment-specific values

You can reuse the same root configuration with different variable files.

For example, define variables in `variables.tf`:

```
variable "application_name" {
  type = string
}

variable "environment" {
  type = string
}
```

Create `development.tfvars`:

```
application_name = "orders-api"
environment      = "development"
```

Create `staging.tfvars`:

```
application_name = "orders-api"
environment      = "staging"
```

Generate environment-specific plans:

```
terraform plan -var-file="development.tfvars"
terraform plan -var-file="staging.tfvars"
```

These commands use the same root configuration. They do not, by themselves, guarantee separate state.

If these environments represent separate infrastructure, ensure the backend and state arrangement correctly separates them. Different variable files with the same state can cause Terraform to propose changes to the same managed resources.

15. Review module changes carefully

Suppose a module's default database size changes.

That change might affect every environment using the module, including production.

Before merging a module change:

1. Identify every environment that uses the module.

2. Determine whether the changed default affects existing callers.
3. Generate plans for affected environments.
4. Inspect replacements and destructive actions.
5. Verify security and availability implications.
6. Review the change with the appropriate owners.
7. Roll out according to the team's deployment policy.

A reusable module increases consistency, but a poorly reviewed module change can also increase the blast radius of a mistake.

16. State commands and resource addresses

Terraform identifies resources through addresses.

For example:

```
terraform_data.application
module.application.terraform_data.application
```

These addresses are different because one resource is declared in the root module and the other inside a child module.

A refactor that moves a resource into a module can change its Terraform address even when the intended real infrastructure remains the same.

Terraform supports state-management operations to handle certain refactors and address changes. For supported configurations, `moved` blocks can document resource address changes.

Conceptual example:

```
moved {
  from = terraform_data.application
  to   = module.application.terraform_data.application
}
```

The destination must correspond to the resource's actual new address.

Review the plan after a refactor. The goal is to avoid accidentally destroying and recreating a resource merely because its configuration location changed.

17. Git practices for Terraform repositories

Infrastructure code deserves the same disciplined review as application code.

A repository might contain:

```
infrastructure/
■■■ main.tf
■■■ variables.tf
■■■ outputs.tf
■■■ versions.tf
■■■ modules/
■■■ environments/
■■■ README.md
```

Use `.gitignore` to exclude local artifacts such as:

```
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
crash.log
```

Review these exclusions against your team's actual workflow. Do not blindly ignore files that the repository intentionally tracks, and never commit state or plan files without a deliberate, approved reason.

Terraform lock files are different: the provider dependency lock file, commonly `.terraform.lock.hcl`, is generally committed so teams can reproduce provider selections.

A useful pull request should explain:

- What infrastructure is changing.
- Which environments are affected.
- Whether resources are created, updated, or replaced.
- Any cost or availability implications.
- Security changes.
- Recovery or rollback considerations.

Layer 3 — Production Engineering & Interview Mastery

18. State migration: local to remote

Scenario: A prototype begins as a local Terraform project. The team now needs a shared workflow.

A safe migration requires more than adding a backend block.

Recommended process

1. Stop other Terraform operations for the environment.
2. Back up the existing state securely.
3. Configure the intended remote backend.
4. Run `terraform init` and carefully review migration prompts.
5. Confirm the migration completed.
6. Verify expected resources with `terraform state list`.
7. Generate a fresh plan.
8. Investigate any unexpected create, delete, or replacement actions.
9. Resume the approved workflow only after verification.

If the plan proposes recreating resources that should already be managed, stop and investigate the state association before applying.

19. Production incident: Terraform wants to recreate existing resources

Scenario: After a module refactor, Terraform proposes replacing an application resource.

Possible causes include:

- A resource address changed.
- A provider attribute requires replacement.
- The resource was removed from state.
- The configuration now points at a different environment.
- A module input changed.
- Provider behavior or versions changed.

Investigation

```
terraform state list
terraform plan
terraform providers
```

Inspect the proposed actions and relevant resource configuration. If necessary, review provider documentation for attributes that force replacement.

Do not automatically use `terraform state rm` to silence the difference. Removing a resource from state changes Terraform's tracking; it does not delete the actual object, and the next plan may attempt to create another one.

Use a supported state move or migration strategy when the issue is a legitimate address refactor.

20. Production incident: state lock cannot be acquired

Scenario: A pipeline fails because another operation appears to hold the state lock.

Possible explanations:

- Another apply is running.
- A previous operation did not release its lock.
- A pipeline was interrupted.
- The backend reports a stale lock.

Response

1. Identify the backend and the reported lock.
2. Check the deployment pipeline and team activity.
3. Verify whether any Terraform operation is still active.
4. Follow the backend's documented stale-lock recovery procedure.
5. Use force-unlock only when you have verified that the lock belongs to an operation that is no longer active and you understand the consequences.
6. Re-run the plan against current state.

Never use force-unlock as a routine retry button.

21. Production incident: an engineer applied to the wrong environment

Scenario: A developer intended to update staging but selected production configuration.

Prevention controls include:

- Separate environment directories and state boundaries where appropriate.
- Explicit environment naming in plans and pipeline output.
- Protected production apply workflows.
- Least-privilege credentials.
- Required review and approval.
- Automated checks for the expected workspace or backend.
- Clear ownership and deployment procedures.

If the wrong change was applied:

1. Establish the actual change and impact.
2. Stop additional changes.
3. Determine whether a corrective change or rollback is safe.
4. Preserve the relevant plan, logs, and audit evidence.
5. Restore the intended configuration through a reviewed workflow.
6. Verify real infrastructure and application behavior.
7. Add a guardrail to prevent recurrence.

Do not assume that reverting a Git commit automatically reverses an infrastructure change.

22. Production incident: sensitive data appears in state

Scenario: A state artifact containing credentials is accidentally shared.

Treat this as a potential secret exposure.

1. Restrict further access to the artifact.
2. Determine which values were exposed and who could access them.
3. Rotate or revoke credentials where appropriate.
4. Review relevant access logs.
5. Secure state and plan storage.
6. Remove unintended copies where possible, while following retention and incident-response requirements.
7. Prevent recurrence through secret-management improvements and access controls.

Removing a value from the current configuration may not remove every historical copy from state versions, backups, plans, or logs.

23. Production architecture exercise: development, staging, and production

Design an infrastructure repository for a Spring Boot backend backed by PostgreSQL.

Your design should address:

- A reusable application module.
- A reusable database module.
- Shared networking patterns.

- Explicit environment-specific variables.
- Separate state arrangements appropriate to each environment.
- Least-privilege access.
- A reviewed plan before applying.
- Safe handling of credentials.
- A recovery strategy for state.
- A migration strategy for changes that require resource replacement.

Questions to answer

1. Which values should be module inputs?
2. Which values should be outputs?
3. Which resources should be shared, and which should be isolated?
4. How will you prevent a staging plan from targeting production?
5. What happens if an engineer loses local state?
6. How will you review a change that replaces a database?
7. How will you detect manual changes to managed infrastructure?

There is no universal folder layout or backend choice. The design should match the platform, team's access model, and operational requirements.

24. SDE-1 interview questions and answers

1. Why does Terraform need state?

State maps Terraform resource addresses to real objects and records information needed to manage those objects across operations.

2. What is remote state?

State stored using a configured remote backend rather than only in a local state file.

3. Does remote state always provide locking?

No. Locking and concurrency behavior depend on the backend and its configuration.

4. What is a Terraform module?

A reusable collection of Terraform configuration with inputs and outputs.

5. What is the difference between a root module and a child module?

The root module is the configuration in the working directory used for a Terraform operation. A child module is called by another module.

6. Are Terraform workspaces a complete production isolation mechanism?

No. They separate state instances, but do not automatically separate credentials, accounts, permissions, or deployment approval policies.

7. Why can moving a resource into a module cause a problem?

Its Terraform address may change. Without a supported migration or move strategy, Terraform may propose creating or destroying resources unnecessarily.

8. What is a `moved` block used for?

It records a supported resource address migration so Terraform can recognize that a resource's address changed without treating it as an unrelated object.

9. What should you do when a plan unexpectedly replaces a database?

Stop, identify why replacement is required, evaluate data and availability risks, correct the configuration or migration strategy, and review a new plan before applying.

10. Why should state not be edited manually?

Direct edits can corrupt resource mappings and create inconsistencies. Use supported state-management operations and recovery procedures.

11. What is state locking?

A coordination mechanism that helps prevent conflicting operations against the same state when supported by the backend.

12. Why is `terraform state rm` potentially dangerous?

It removes a resource from Terraform's state tracking without deleting the real object. A subsequent plan may propose creating another resource or otherwise produce unexpected changes.

13. What is infrastructure drift?

A difference between real infrastructure and the configuration or state Terraform expects.

14. Should development and production share the same state?

Usually they should have separate state boundaries when they represent independently managed environments. The exact arrangement depends on architecture and access requirements.

15. What is the purpose of `.terraform.lock.hcl`?

It records provider dependency selections and checksums to support more reproducible provider installation. It is generally committed to version control.

16. Why can a module default change be risky?

Every caller that inherits the default may be affected, potentially changing resource sizes, security settings, or lifecycle behavior across environments.

17. How should secrets be handled in Terraform?

Use approved identity and secret-management mechanisms, restrict state and plan access, and avoid committing secret values. Sensitive marking alone does not prevent persistence.

18. Does reverting a Terraform configuration commit reverse infrastructure changes?

No. It changes source control. A new plan and apply may be required to reconcile infrastructure, and the correction must account for irreversible changes and data compatibility.

19. How would you migrate local state to a remote backend?

Stop concurrent operations, back up state, configure the backend, initialize and review migration, verify state contents, and inspect a fresh plan before resuming changes.

20. How do you prevent applying to the wrong environment?

Use clear environment boundaries, protected deployment workflows, least-privilege credentials, explicit plan review, and automated environment verification.

25. Hands-on mastery assessment

Use the local Terraform exercise from Chunk 1 and extend it.

- [ ] Create an `application` child module.
- [ ] Define module inputs and outputs.
- [ ] Call the module from a root configuration.
- [ ] Create separate development and staging variable files.
- [ ] Explain why separate variable files do not automatically imply separate state.
- [ ] Inspect state with `terraform state list`.
- [ ] Explain why state and plan files must be protected.
- [ ] Add a resource address migration using a suitable `moved` block in a controlled example.
- [ ] Run `terraform fmt -recursive`.
- [ ] Run `terraform validate`.
- [ ] Generate and review a plan.
- [ ] Describe a safe local-to-remote state migration.
- [ ] Explain how to investigate an unexpected database replacement.
- [ ] Describe how to recover from a stale lock safely.

Mastery standard

You should be able to design a reusable module interface, explain the difference between state and real infrastructure, separate environments safely, diagnose state-related failures, and defend an infrastructure change during a code review.

Terraform Chunk 2 completion protocol

Save this chapter as:

`content/terraform/02-terraform-state-modules-environments-safe-workflows.md`

Terraform has three planned content chunks. Do not generate a PDF yet.

The final Terraform handbook must combine all three complete chapters into one PDF:

`docs/terraform/BEH_TERRAFORM_COMPLETE_HANDBOOK.pdf`

After all three chapters are saved and verified:

1. Build the complete Terraform handbook PDF.
2. Verify chapter order, code blocks, tables, and page rendering.
3. Update the README with verified progress.
4. Commit the completed Terraform sources and verified PDF with the exact commit message:

`Terraform done`

Do not create a PDF for this chapter alone. The repository's final deliverable remains 13 individual tool handbooks and one complete master handbook.

Part III: Production Infrastructure, Security, CI/CD, Reliability & Incident Mastery

Chunk 3 of 3 · Backend Engineering Handbook

Terraform — Chunk 3 of 3: Production Infrastructure, Security, CI/CD, Reliability & Incident Mastery

Backend Engineering Tools & Infrastructure Handbook

Source file:
`content/terraform/03-terraform-production-security-cicd-reliability-incident-mastery.md`

Learning objective: Learn to operate Terraform as a production engineering tool: secure infrastructure, manage changes through CI/CD, reduce deployment risk, control costs, and investigate incidents involving failed or dangerous infrastructure changes.

Layer 1 — Fundamentals & Mental Models

1. Terraform in production: from configuration to controlled change

In development, Terraform helps you create infrastructure. In production, it also becomes part of your organization's change-management system.

A production infrastructure change should answer five questions:

1. What is changing?
2. Why is it changing?
3. What could break?
4. Who is authorized to approve and apply it?
5. How will we verify and recover from the change?

The production workflow is therefore more than `terraform apply`.

```
Engineer changes configuration
            |
            v
     Pull request created
            |
            v
 Formatting and validation
            |
            v
      Plan generated
            |
            v
   Impact and risk review
            |
            v
    Authorized approval
            |
            v
     Approved plan applied
            |
            v
 Infrastructure verification
            |
            v
 Monitoring and audit trail
```

This workflow reduces the chance of an engineer accidentally deleting a database, exposing a network service, or changing the wrong environment.

Terraform supplies infrastructure-management mechanisms. Your team must provide appropriate access controls, review policies, operational procedures, and monitoring.

2. Understand the blast radius

The **blast radius** is the scope of damage or disruption a change can cause.

Examples:

- Changing one development resource has a relatively limited scope.
- Changing a shared network can affect multiple services.
- Changing a production database can affect business-critical data.
- Changing a widely reused module may affect many environments.

A good infrastructure design limits unnecessary blast radius through:

- Separate environment boundaries.
- Narrowly scoped identities and permissions.
- Reusable modules with controlled interfaces.
- Small, reviewable changes.
- Explicit ownership of resources.
- Carefully planned database and networking migrations.
- Tested recovery procedures.

A Terraform plan can reveal which resources may change, but it does not automatically understand every business dependency. Review the plan in the context of the actual architecture.

3. Pin provider dependencies and manage upgrades

Terraform configurations depend on providers, and provider versions can influence validation, planning, and resource behavior.

A typical provider requirement looks like this:

```
terraform {
  required_providers {
    example = {
      source  = "example-org/example"
      version = "~> 2.4"
    }
  }
}
```

This is an illustrative provider declaration; the source and version constraint must match a real provider.

The version constraint defines which versions Terraform is allowed to select. The dependency lock file, `.terraform.lock.hcl`, records the selected provider versions and checksums for reproducibility.

A safer upgrade workflow

1. Identify the provider upgrade and its purpose.
2. Review release notes and breaking changes.
3. Update the version constraint deliberately.
4. Run `terraform init -upgrade` in a controlled branch.
5. Review changes to `.terraform.lock.hcl`.
6. Run formatting and validation.
7. Generate plans for affected environments.
8. Test in development or staging.
9. Apply through the approved release workflow.

Avoid upgrading all providers during an unrelated production change unless there is a deliberate reason to combine those changes.

Reproducible provider selection makes reviews more meaningful because reviewers can understand which provider implementation is being used.

4. Production infrastructure security

Terraform can create security controls, but the configuration must express the intended policy correctly.

Consider the security of a backend application that needs to connect to PostgreSQL.

A reasonable architecture might be:

```
Internet
   |
   v
Public entry point
   |
   v
Backend application
   |
   v
Private database network
   |
   v
PostgreSQL
```

The exact implementation depends on the hosting platform. The important principle is that the database should not be exposed publicly without a justified requirement and suitable controls.

Security responsibilities include:

- Network segmentation.
- Least-privilege identities.
- Encryption in transit and at rest where appropriate.
- Secret management.
- Auditability.
- Resource ownership.
- Secure defaults.
- Controlled infrastructure changes.

Do not confuse creating a network resource with establishing a secure network. Firewall rules, routes, identities, service exposure, and platform-specific policies must also be considered.

5. Secrets and identity in production workflows

Infrastructure deployments need credentials or identities to interact with their target platforms.

Safer approaches include:

- Short-lived credentials.
- Federated workload identity where supported.
- Platform-native managed identities.
- Least-privilege permissions.
- Dedicated deployment identities.
- Centralized secret-management systems.

Avoid hard-coding access keys in Terraform files or committing credentials to Git.

Example of a pattern to avoid:

```
provider "example" {
  access_key = "hard-coded-secret"
}
```

The provider and attribute are illustrative. The problem is the hard-coded credential, not the specific provider syntax.

Prefer an approved identity mechanism supported by the selected provider. Environment variables can be useful for some workflows, but long-lived environment-variable credentials still need secure sourcing, restricted access, and rotation.

Marking a value as `sensitive` may redact it from certain displays. It does not guarantee that the value will never appear in state, saved plans, logs, or other artifacts.

6. Policy as Code and preventive guardrails

Policy as Code means expressing infrastructure rules in machine-checkable form.

Examples of policies:

- Production resources must include ownership tags.
- Public access to databases is prohibited unless explicitly approved.
- Storage resources must meet encryption requirements.
- Resources must use approved regions.
- Required monitoring settings must be present.
- Destructive changes require additional review.

Policy enforcement can be implemented using platform-native controls, CI checks, policy frameworks, or custom validation.

The purpose is to catch unacceptable changes before they reach production.

Prevention versus detection

Control	Example
Preventive	Reject a plan that violates an approved security policy
Detective	Alert when an unexpected public endpoint appears
Corrective	Apply a reviewed configuration change to restore the intended rule

All three can be useful. Prevention reduces risk, detection identifies failures that escape prevention, and correction restores the intended condition.

7. Infrastructure reliability and availability

Terraform can provision resources that support reliability, such as:

- Multiple application instances.
- Load balancers.
- Health checks.
- Private networking.
- Database replicas.
- Backups and recovery infrastructure.
- Monitoring systems.

But provisioning a resource does not prove that the resulting system meets its reliability objectives.

A production design must consider:

- Failure domains.
- Capacity and scaling.
- Dependency limits.
- Data durability.
- Recovery time.
- Recovery point.
- Operational access.
- Monitoring and alerting.
- Application-level behavior.

For example, creating two application servers does not provide meaningful high availability if both depend on a single database with no suitable recovery plan.

Terraform manages infrastructure configuration. Application resilience and operational procedures remain essential.

8. Cost management and resource lifecycle

Infrastructure cost is a production concern.

Common sources of unnecessary cost include:

- Oversized compute resources.
- Forgotten development environments.
- Unused disks or snapshots.
- Excessive logging retention.
- Unnecessary public endpoints.
- Overprovisioned databases.
- Duplicate resources caused by poor state management.

Cost controls may include:

- Resource tagging.
- Budget alerts.
- Environment-specific sizing.
- Scheduled cleanup for disposable environments.
- Reviewing planned resource creation.
- Periodic utilization reviews.
- Clear ownership and expiration policies for temporary resources.

Terraform can make resource ownership and intended configuration more visible, but it cannot independently determine the optimal size or cost of every resource.

Important: `terraform destroy` is not a general-purpose cost optimization strategy. It can destroy data and business-critical infrastructure. Cleanup should be deliberate and scoped to disposable resources.

Layer 2 — Practical Engineering & Workflows

9. Build a production-minded Terraform repository

A repository might use the following structure:

```
infrastructure/
■■■ versions.tf
■■■ providers.tf
■■■ main.tf
■■■ variables.tf
■■■ outputs.tf
■■■ modules/
■   ■■■ network/
■   ■■■ application/
■   ■■■ database/
■■■ environments/
■   ■■■ development/
■   ■■■ staging/
■   ■■■ production/
■■■ policies/
■■■ README.md
```

This is one possible structure, not a universal requirement.

Keep environment boundaries, provider configuration, modules, and operational documentation understandable. Avoid introducing abstractions that make the effective infrastructure difficult to trace.

For each environment, document:

- The infrastructure it owns.
- The backend and state boundary.
- Required identities and permissions.
- How plans are generated and reviewed.
- How changes are applied.
- How failures are recovered.
- Who owns the environment.

10. Validate configuration before review

A basic local validation workflow:

```
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
```

`terraform init -backend=false` initializes the configuration without initializing its configured backend. It can be useful for validation workflows, but provider installation and initialization requirements still apply.

For a working environment with an initialized backend, generate the plan using the team's normal backend configuration.

```
terraform plan -out=tfplan
terraform show tfplan
```

Review the plan before applying it.

Remember:

- Formatting checks formatting.
- Validation checks configuration consistency.
- Planning previews proposed changes.
- Applying executes changes.
- Post-deployment verification checks the real outcome.

No single step replaces all the others.

11. A practical CI/CD pipeline

A production pipeline can be organized into distinct stages.

Stage A — Pull request validation

Run:

```
terraform fmt -check -recursive
terraform validate
```

Additional checks may include static analysis, security scanning, policy checks, and module tests.

Stage B — Generate a plan

Initialize the correct backend and generate a plan for the intended environment.

```
terraform plan -out=tfplan
terraform show tfplan
```

The pipeline must use the correct environment, identity, provider dependencies, and state.

Stage C — Review

Review:

- Resources to be created.
- Resources to be updated.
- Resources to be replaced or destroyed.
- IAM and network changes.
- Cost implications.
- Database and storage implications.
- Provider upgrades.
- State and backend changes.

Stage D — Apply

Use an authorized workflow to apply the approved changes.

```
terraform apply tfplan
```

Saved plan files can contain sensitive information and should be protected. A saved plan should be applied only within the intended environment and workflow. If relevant configuration, infrastructure, or assumptions have changed, regenerate the plan rather than relying on a stale artifact.

Stage E — Verify

Verify the resulting infrastructure using provider tooling, Terraform outputs where appropriate, application health checks, logs, and monitoring.

A successful apply confirms that Terraform completed its operation; it does not guarantee that the entire application works correctly.

12. Handle failed applies safely

A Terraform apply may fail partway through a multi-resource change.

For example, Terraform may create a network and application resource but fail while creating a database because of a quota or permission issue.

Do not assume that a failed apply means nothing changed.

Use this workflow:

1. Read the error and identify the failed operation.
2. Check the real infrastructure through the provider or platform.
3. Inspect Terraform state.
4. Determine which resources were successfully created or modified.
5. Correct the underlying cause.
6. Generate a fresh plan.
7. Review the new plan before retrying.

Useful commands include:

```
terraform state list
terraform plan
terraform show
```

The state and infrastructure may already reflect some completed operations. A fresh plan is essential for understanding what Terraform now proposes to do.

13. Resource lifecycle and safe deletion

Before deleting an infrastructure resource, determine whether it contains persistent data or is depended on by other systems.

Questions to ask:

- Is the resource stateful?
- Does it contain business-critical data?
- Are backups available and restorable?

- Will dependent resources fail?
- Is there a maintenance window?
- Is the deletion reversible?
- Is a migration safer than replacement?

Terraform lifecycle settings can alter how Terraform handles certain changes. For example, `prevent_destroy` can protect against some accidental destroy plans while the lifecycle rule remains applicable.

Illustrative configuration:

```
resource "example_database" "orders" {
  # Provider-specific database configuration.

  lifecycle {
    prevent_destroy = true
  }
}
```

The resource type is illustrative and is not a deployable database configuration.

`prevent_destroy` is a guardrail, not a backup or recovery strategy. It does not prevent every external deletion path, and removing the resource configuration can change the situation. Review lifecycle behavior for the actual provider and resource.

14. Drift detection workflow

Suppose an engineer manually changes a security rule outside Terraform.

Start with:

```
terraform plan
```

Then:

1. Identify the exact resource and attribute.
2. Confirm the actual platform configuration.
3. Determine why the manual change occurred.
4. Decide whether the intended configuration should change or the external modification should be reversed.
5. Generate a new plan after resolving the discrepancy.
6. Apply through the approved workflow.
7. Consider automated drift detection for important environments.

A plan is useful for identifying differences, but not every external change is necessarily represented identically by every provider or resource. Understand the resource's behavior before relying on drift detection as a complete security control.

15. Logging and auditing infrastructure changes

When an infrastructure incident occurs, the team should be able to answer:

- Who requested the change?
- Who approved it?

- Which plan was reviewed?
- Which identity applied it?
- Which provider and configuration versions were used?
- What changed in the target platform?
- Was the application healthy afterward?

Keep useful evidence such as:

- Pull request references.
- CI/CD execution identifiers.
- Plan review records.
- Terraform logs where appropriate.
- Platform audit events.
- Post-deployment verification results.

Do not publish secrets or sensitive state contents in logs. Operational evidence should be accessible to authorized responders without unnecessarily exposing credentials.

16. Module testing and reusable infrastructure

Reusable modules need validation just like application libraries.

Useful checks include:

- Formatting and validation.
- Reviewing required and optional inputs.
- Verifying outputs.
- Checking default values.
- Testing expected resource relationships.
- Generating plans for representative configurations.
- Testing security policy requirements.
- Reviewing upgrades against all relevant environments.

Where practical, use automated infrastructure tests in disposable environments. Avoid running destructive tests against shared or production infrastructure.

A module change that works in development can still behave differently in production because of quotas, network boundaries, provider differences, or environment-specific inputs.

Layer 3 — Production Engineering & Interview Mastery

17. Incident simulation: a Terraform apply fails halfway through

Scenario: A deployment creates some resources, then fails while creating the database.

Step 1 — Stop and assess

Do not immediately rerun the pipeline repeatedly. Determine whether the failed operation left infrastructure behind.

Step 2 — Inspect the error

Check for:

- Permission failures.
- Resource quotas.
- Invalid arguments.
- Network conflicts.
- API rate limits.
- Provider errors.
- Dependency failures.

Step 3 — Inspect state and actual infrastructure

```
terraform state list
terraform plan
```

Check the platform itself to confirm which objects exist and whether they are healthy.

Step 4 — Correct the cause

Fix the configuration, permissions, quota, or environmental issue. Do not remove state entries merely to make the next plan appear clean.

Step 5 — Plan again

Generate a fresh plan and review it for unexpected creation, replacement, or destruction.

Step 6 — Apply and verify

Use the approved deployment process and verify the resulting infrastructure and application behavior.

Interview principle: A failed apply is a partially completed operation until evidence proves otherwise.

18. Incident simulation: a Terraform change exposes a database

Scenario: A pull request changes network rules, and the resulting database becomes publicly reachable.

Immediate response

1. Establish the exposure and affected resource.
2. Restrict public access using an approved emergency procedure.
3. Confirm that authorized application traffic still works.
4. Review platform audit events and the Terraform change.
5. Determine whether credentials or data may have been exposed.
6. Rotate credentials if exposure warrants it.

7. Restore the intended network configuration through a reviewed change.
8. Add a preventive policy and a regression check.

Do not wait for a normal release window if there is an active security exposure and an authorized emergency mitigation is available.

The corrective Terraform configuration should reflect the intended security posture so that a later apply does not reintroduce the exposure.

19. Incident simulation: a provider upgrade changes resource behavior

Scenario: A provider upgrade produces an unexpected replacement in a production plan.

Response

- Stop the production apply.
- Inspect the provider version and lock-file change.
- Identify the resource attribute causing replacement.
- Review the provider's upgrade notes.
- Compare plans under the previous and proposed versions where safely possible.
- Test in a non-production environment.
- Decide whether to modify the configuration, defer the upgrade, or follow a migration procedure.
- Apply only after the replacement risk is understood and approved.

Provider upgrades should be treated as infrastructure changes, not as harmless dependency maintenance.

20. Incident simulation: unexpected infrastructure cost

Scenario: The cloud bill increases after a Terraform deployment.

Investigate:

1. Identify which resources changed during the release.
2. Compare planned and actual resource configurations.
3. Inspect utilization, capacity, and scaling behavior.
4. Check for duplicate resources, unused disks, or unexpected replicas.
5. Confirm whether a module default or provider behavior changed.
6. Identify whether the increase is expected, temporary, or erroneous.
7. Propose a reviewed correction.

Do not automatically destroy resources just because they appear expensive. Confirm ownership, dependencies, data retention, and recovery implications before making changes.

21. Production readiness checklist

Configuration and dependencies

- [ ] Terraform and provider requirements are explicit.
- [ ] Provider lock-file changes are reviewed.

- [ ] Configuration is formatted and validated.
- [ ] Reusable modules have documented inputs and outputs.
- [ ] Resource dependencies are clear.
- [ ] Destructive changes are reviewed carefully.

State and recovery

- [ ] Shared environments use an appropriate remote-state design.
- [ ] State access is restricted.
- [ ] Locking or concurrency behavior is understood.
- [ ] State recovery options are documented and tested where feasible.
- [ ] State and plan artifacts are protected.
- [ ] State migration and recovery procedures are documented.

Security

- [ ] Credentials are not hard-coded.
- [ ] Deployment identities follow least privilege.
- [ ] Network exposure is intentional.
- [ ] Database access is appropriately restricted.
- [ ] Required encryption controls are configured.
- [ ] Secret rotation procedures exist.
- [ ] Policy checks catch important prohibited configurations.

Delivery and operations

- [ ] Plans are reviewed before production apply.
- [ ] Production deployment permissions are restricted.
- [ ] Failed applies are handled through a documented process.
- [ ] Post-deployment checks verify actual infrastructure.
- [ ] Drift detection is appropriate for the environment.
- [ ] Audit evidence is retained securely.
- [ ] Cost and resource ownership are visible.
- [ ] Incident response and recovery procedures are understood.

---

22. SDE-1 interview questions and answers

1. Why should production infrastructure changes go through code review?

Review makes the proposed changes visible before execution and helps identify destructive, insecure, or unexpectedly expensive operations.

2. What is the blast radius of an infrastructure change?

The scope of systems and users that could be affected if the change fails or behaves unexpectedly.

3. Why pin provider dependencies?

Explicit version constraints and a committed lock file help make provider selection reproducible and upgrades deliberate.

4. Does a successful Terraform apply prove the application is healthy?

No. It shows that Terraform completed its planned operations. Application behavior, dependencies, networking, and monitoring must still be verified.

5. What should you do if an apply fails halfway through?

Inspect the error, state, and real infrastructure; correct the cause; generate a fresh plan; review it; and apply through the approved workflow.

6. Why is a saved plan sensitive?

It may contain infrastructure details and sensitive values. Protect it from unauthorized access and avoid committing it.

7. What is Policy as Code?

The practice of expressing infrastructure rules in machine-checkable form so violations can be detected or prevented automatically.

8. What is the difference between preventive and detective controls?

Preventive controls stop prohibited changes before they happen. Detective controls identify conditions that already exist or have occurred.

9. How does Terraform relate to infrastructure security?

It can manage security-relevant infrastructure configuration, but safe outcomes require correct policies, identities, network rules, secret handling, and review processes.

10. Why is `prevent_destroy` not a backup strategy?

It is a configuration-level guardrail against certain Terraform destroy operations. It does not preserve data or prevent all external deletion paths.

11. What should you do if a provider upgrade introduces unexpected resource replacement?

Stop the risky apply, inspect the version and plan, review provider changes, test in a safe environment, and proceed only after the replacement implications are understood.

12. How do you detect infrastructure drift?

Generate plans and use suitable platform monitoring or drift-detection workflows to compare actual resources with the intended configuration.

13. Why should infrastructure changes be small and reviewable?

Smaller changes are easier to understand, test, attribute, and recover from, reducing uncertainty and blast radius.

14. How would you respond to a publicly exposed database?

Restrict access through an authorized mitigation, establish impact, review audit evidence, rotate credentials if warranted, restore the intended configuration, and add preventive controls.

15. How should infrastructure credentials be provided to CI/CD?

Prefer short-lived or federated identities where supported, with least-privilege permissions and secure credential handling.

16. Why is state not a complete infrastructure backup?

It tracks Terraform-managed resource mappings and attributes; it does not necessarily preserve resource contents or provide everything needed to recover the full system.

17. What should you verify after an infrastructure deployment?

Confirm the resulting resources, network access, application health, dependency connectivity, monitoring signals, and any expected changes in cost or capacity.

18. How would you investigate an unexpected cost increase?

Correlate the increase with deployment changes, inspect resource sizing and utilization, identify unexpected resources, and implement a reviewed correction.

19. Why can a shared module change be risky?

Many environments may inherit the same change. A default or resource change can affect multiple services and increase the blast radius.

20. What does production readiness mean for Terraform?

The configuration, state management, security, deployment controls, observability, and recovery procedures are designed and tested well enough to manage infrastructure safely.

---

23. Final Terraform capstone

Scenario

You are responsible for infrastructure supporting a Spring Boot Orders API backed by PostgreSQL.

Design a Terraform workflow that provisions the required infrastructure and supports safe future changes.

Your solution should include:

1. A reusable network module.
2. An application infrastructure module.
3. A database infrastructure module.
4. Separate environment configurations or state boundaries appropriate to the deployment model.
5. A documented backend and state-recovery approach.
6. Explicit provider version requirements.
7. A secure deployment identity.
8. Restricted database access.
9. A reviewed CI/CD plan-and-apply workflow.
10. A post-deployment verification checklist.
11. Drift detection and incident-response procedures.
12. A strategy for handling database replacement and schema migration risk.

Required failure drills

Perform these only in a disposable test environment.

- Introduce a configuration validation error and verify that the pipeline stops.
- Change a harmless resource input and review the plan.

- Simulate an unexpected replacement and practice stopping the deployment.
- Introduce an intentional drift in a disposable resource and inspect the plan.
- Test a failed apply and document how you determine which resources exist.
- Verify that unauthorized identities cannot apply production changes.
- Document how to recover if the state backend becomes unavailable.

Final mastery standard

You should be able to explain not only how Terraform creates infrastructure, but how an engineering team can manage that infrastructure securely and reliably throughout its lifecycle.

Terraform completion protocol

This is the **third and final planned Terraform content chunk**.

Save this chapter as:

`content/terraform/03-terraform-production-security-cicd-reliability-incident-mastery.md`

After all three Terraform source chapters have been saved and verified:

1. Confirm that all three Markdown files contain their complete chapters.
2. Build one complete Terraform handbook PDF from all three chapters in curriculum order.
3. Save it as `docs/terraform/BEH_TERRAFORM_COMPLETE_HANDBOOK.pdf`.
4. Verify chapter order, code blocks, tables, links, and page rendering.
5. Update the README with verified source and PDF status.
6. Commit the complete Terraform source chapters and verified PDF using the exact commit message:

```
Terraform done
```

Do not generate a PDF for this chapter alone.

Handbook project milestone

The planned curriculum is now complete at the *content-authoring level in this conversation* for Terraform. This does not confirm that the chapters have been saved to the repository, that any PDF has been generated, or that any Git commit has been made.

The overall project still requires all 13 individual tool PDFs and the final master handbook:

`docs/BEH_MASTER_BACKEND_ENGINEERING_HANDBOOK.pdf`

The master must contain all 13 complete tool handbooks, the end-to-end integration lesson, and the final readiness checklist.